AI-Enabled Developer

Module C – AI-assisted development

Level 1: Foundations · Topics B14–B19

What changes when the model can run your tests and edit your files. Every slide carries a worked example — a real agent trace, real instruction-file lines, a real plan with real rejections, and a real test.

6	210	6	10
topics	minutes	concept diagrams	role profiles

© 2026 All rights reserved by Smart Working Solutions Ltd.

LEVEL 1: FOUNDATIONS

Module C at a glance

Six topics, six worked examples, roughly three and a half hours. Module B taught you to control a single request. Module C is what changes when the model can act.

B14	**The agent loop** Worked example · full width	25 min
B15	**Repo instruction files** Worked example · full width	35 min
B16	**Plan-then-execute** Worked example · full width	45 min
B17	**Spec-driven development** Worked example · full width	40 min
B18	**Agent-assisted TDD** Worked example · full width	40 min
B19	**Tool landscape and selection** Worked example · full width	25 min

Role chips on every topic slide: FS Full-stack · BE Backend · FE Frontend · QA · DO DevOps/SRE · DE Data Eng · DS Data Sci/ML · PB Product/BA · SEC Security · EM Tech Lead

© 2026 All rights reserved by Smart Working Solutions Ltd.

The agent loop

OUTCOME Explain perceive-plan-act-observe and identify the stop condition.

- **Perceive → plan → act → observe → repeat, until a stop condition.** That is an agent. Everything else is configuration.
- The trace on the right is real-shaped: it edited, ran the tests, read a failure, and fixed the rounding on its own.
- That feedback loop is the whole difference from autocomplete — it has a signal, not just a prior.
- **Agent = Model + Harness.** The model is rented; the harness — context, tools, tests, permissions — is yours.

ONE REAL LOOP — THE AGENT'S OWN TRACE

PERCEIVE	`read checkout.py, discount.py, orders schema`
PLAN	`apply code in Order.total() before tax`
ACT	`edit discount.py (+14 −11)`
OBSERVE	`ran pytest → 3 failed: rounding on line 88`
ACT	`edit discount.py → use Decimal, not float`
OBSERVE	`ran pytest → all 41 pass`
STOP	`tests green, task met`

EXERCISE Watch an agent run on a real ticket and annotate each loop iteration live — including what finally stopped it.

ROLES: FS BE FE QA DO DE DS PB SEC EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Repo instruction files

OUTCOME Write an AGENTS.md that names a specific failure it prevents.

- The file carries what the agent cannot infer: exact commands, conventions, never-touch paths, definition of done.
- The left file is real — and worthless. Every line is true of every repo, so it constrains nothing.
- The right file is the same length and prevents five specific bugs. Line one stops the rounding failure from B14.
- Same content, different filenames per tool — write it once, translate it.

TWO REAL AGENTS.md FILES — SAME REPO

HOROSCOPE — prevents nothing

```
# AGENTS.md

- Write clean, maintainable code
- Follow best practices
- Be careful with the database
- Handle errors appropriately
- Use good naming conventions
```

True of every repo ever — so it tells the agent nothing about yours

SPECIFIC — each line names a real bug it stops

```
# AGENTS.md

Money is Decimal. Never float.
All queries scoped by tenant_id.
Never edit migrations/ or vendor/.
Test one file: pytest path::name
One ticket, one change. No new deps.
```

Line 1 alone prevents the rounding bug from the trace above

THE TEST FOR EVERY LINE: name the specific failure it prevents. If you can't, you've written a horoscope — delete it.

EXERCISE Write five lines of AGENTS.md for a real repo you own. Every line must name a failure it prevents.

ROLES: FS BE FE QA DO DE DS PB SEC EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Plan-then-execute

OUTCOME Review and edit an agent's plan before any code is written.

- **Never let it write code before you have read and edited the plan.** The highest-return habit in the course.
- The plan on the right is real: six steps, two rejected — a needless migration, and a scope-creep refactor.
- Both rejections cite a reason from the instruction file. The reasoning is the teaching, not the edit.
- **The plan is where you have leverage. The diff is where you have homework** — as the cost bars show.

A REAL PLAN, REVIEWED — TWO STEPS REJECTED IN 90 SECONDS

AGENT'S PLAN for "add discount codes to checkout"

```
✓  1. Add DiscountCode model
✓  2. Add /apply-code endpoint
✗  3. Add discount_pct column to orders (migration)   ← instruction file forbids migrations
✓  4. Apply code in Order.total()
✓  5. Add tests for expiry + invalid code
✗  6. Refactor the pricing module while here          ← scope creep — one ticket, one change
```

COST OF CATCHING IT



EXERCISE Get a plan for a real ticket. Edit it by hand and record two things you rejected and why.

ROLES: FS BE FE QA DO DE DS PB SEC EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Spec-driven development

OUTCOME Write acceptance criteria precise enough to be executed rather than interpreted.

- A vague ticket now produces four hundred lines of confident, wrong code that passes its own tests.
- The left panel shows two agents diverging from the same ticket — both defensible, both built.
- The spec on the right converges them: testable criteria, concrete edge examples, and explicit non-goals.
- **Ambiguity used to cost a conversation. It now costs a sprint.**

THE SAME FEATURE — A TICKET, THEN A SPEC AN AGENT CAN EXECUTE

VAGUE TICKET

```
"Add discount codes to checkout.
 Should handle expiry."
```

Two agents build two different products. Both defensible.

```
Agent A: percentage discounts, stacking
  allowed, expiry = server date
Agent B: fixed-amount only, no stacking,
  expiry = midnight UTC, 410 on expired
```

SPEC — criteria, examples, non-goals

```
CRITERIA
expired code → HTTP 410, cart unchanged
valid code → fixed £ off, min 0 total
EXAMPLES
code at 23:59:59 on expiry date = valid
code for deleted product → 404
NON-GOALS
no percentage codes · no stacking
do not touch the pricing module
```

EXERCISE Rewrite a vague ticket as a spec with examples, edge cases and explicit non-goals. Hand it to a partner's agent and compare.

ROLES: FS · BE · FE · QA · DO · DE · DS · PB · SEC · EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Agent-assisted TDD

OUTCOME Own the definition of correct by writing the failing test yourself.

- **You** write the failing test. The agent makes it pass. You own the definition of correct.
- The left test has an excellent name and a worthless assertion — it passes even if the discount is ignored.
- The right test asserts the actual behaviour: total is 90, discount is 10. It fails when the code is wrong.
- **Read the assertions, not the test names.** A green check that tests nothing switches off your suspicion.

WHO WROTE THE TEST? — READ THE ASSERTION, NOT THE NAME

AGENT WROTE CODE + TEST — theatre

```
def test_discount_applies_correctly():
    order = make_order(total=100)
    result = apply("SAVE10", order)
    assert result is not None

# green tick. tests nothing.
# passes even if SAVE10 is ignored.
```

Excellent name. The assertion checks it returned "something".

YOU WROTE THE TEST — real oracle

```
def test_save10_takes_ten_off():
    order = make_order(total=100)
    result = apply("SAVE10", order)
    assert result.total == 90
    assert result.discount == 10

# fails if the code does not apply.
```

You own the definition of correct. It fails when the code is wrong.

Never the same agent for the code and its test. That is not testing — it is a confession co-signed by the defendant.

EXERCISE Write the failing acceptance test by hand, then let the agent implement against it. Then audit an AI-generated test file for theatre.

ROLES: FS, BE, FE, QA, DO, DE, DS, PB, SEC, EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Tool landscape and selection

OUTCOME Choose between terminal, IDE, async-cloud and open-weight tools on stated criteria.

- Four categories, six criteria. The table scores them for one real context: a regulated backend team.
- For that team, "security sign-off" is the deciding row — the async cloud option is simply blocked.
- Offline open-weight wins here not on capability but on approval and data residency.
- Any ranking is stale within a quarter. **Score the criteria for your own context; name the switching condition.**

FOUR CATEGORIES SCORED FOR ONE REAL CONTEXT (a regulated backend team)

	Terminal agent	AI-native IDE	Async cloud	Open-weight, offline
Harness depth	deep	medium	medium	deep
Model per task	yes	some	yes	yes
Data residency	vendor	vendor	vendor	your infra
Security sign-off	pending	pending	blocked	approved

For THIS team, security sign-off decides it. The best harness you can't get approved is worth nothing. Switch if the hosted tools get cleared.

EXERCISE Score three tools against the six criteria for your own context, then name the condition under which you would switch.

ROLES: FS | BE | FE | QA | **DO** | DE | DS | PB | SEC | EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

PLANNING REFERENCE

Module C exercises

Exercises live here rather than on each topic slide. Each is repeated in the speaker notes of its topic.

ID	Topic	Exercise	Min
B14	**The agent loop**	Watch an agent run on a real ticket and annotate each loop iteration live — including what finally stopped it.	25
B15	**Repo instruction files**	Write five lines of AGENTS.md for a real repo you own. Every line must name a failure it prevents.	35
B16	**Plan-then-execute**	Get a plan for a real ticket. Edit it by hand and record two things you rejected and why.	45
B17	**Spec-driven development**	Rewrite a vague ticket as a spec with examples, edge cases and explicit non-goals. Hand it to a partner's agent and compare.	40
B18	**Agent-assisted TDD**	Write the failing acceptance test by hand, then let the agent implement against it. Then audit an AI-generated test file for theatre.	40
B19	**Tool landscape and selection**	Score three tools against the six criteria for your own context, then name the condition under which you would switch.	25

© 2026 All rights reserved by Smart Working Solutions Ltd.

END OF MODULE C

What they can do now — and what comes next

AFTER MODULE C

- Explain the agent loop and identify the stop condition in a live run.
- Write an instruction file where every line names a specific failure it prevents.
- Review and edit an agent’s plan before a single line of code is written.
- Write acceptance criteria precise enough to be executed rather than interpreted.
- Own the definition of correct by writing the failing test themselves.
- Choose tooling on stated criteria rather than on a leaderboard or a tweet.

NEXT: MODULES D & E

What tool use actually means, and grounding with citations — two short bridge modules, both prerequisites for Security.

Module C gave the model the ability to act on your files. Next is understanding where that action actually happens.

Review the plan, not the diff.

© 2026 All rights reserved by Smart Working Solutions Ltd.